Examen T1
Sistemas Operativos

Nombre y apellidos: _____

Pregunta 1 (4 puntos)

Responder a las siguientes cuestiones:

1. (1 p) A nivel de sistema operativo, ¿cuáles son los componentes básicos de los computadores vistos en clase? Pon ejemplos de cada uno de ellos.
2. (1 p) ¿Qué recursos gestiona el sistema operativo? ¿Cómo lo hace?
3. (1 p) ¿Qué servicios ofrecen el sistema operativo y el software del sistema? Pon ejemplos de cada uno de ellos.
4. (1 p) ¿Qué tipos de virtualización existen? ¿En qué se diferencian? Dibuja diagramas para respaldar la respuesta.

1) A nivel de sistemas operativos los componentes básicos són:

- Usuario → personas, ingenieros o incluso otro sistema
- Aplicaciones → software que resuelve el problema del usuario (spotify)
- S.O
  - Software del sistema → Herramientas que facilitan el desarrollo y uso (compiladores, interfaz gráfica...)
  - Núcleo del sistema (kernel) → quien gestiona procesos, memoria, ficheros...
  - Hardware → la parte física (cpu, memoria, discos)

2) El sistema operativo gestiona recursos como: administración de la CPU, memoria y dispositivos. Lo hace como interfaz gráfica, ocultando la complejidad del hardware (abstracción) ofreciendo una máquina virtual más sencilla de usar y programar

Created with Notewise

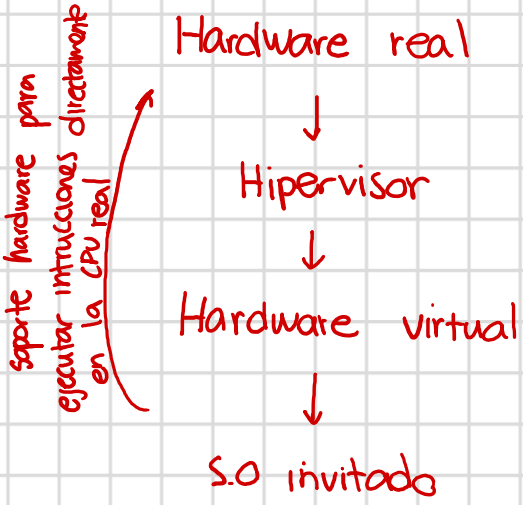
3) Ofrecen servicios como:

- Creación y ejecución de programas (abrir spotify)
- Operación de E/S y manipulación de ficheros (mover un archivo dentro de una carpeta)
- Detección de errores (pantallazo azul)
- Seguridad y control de acceso (contraseñas, system calls)

4) Emulación: simula hardware diferente a través de la traducción de instrucciones

CPU ARM —traducción→ CPU Intel

Virtualización completa: crea una versión virtual de la máquina (hardware y S.O) controlada por el hipervisor



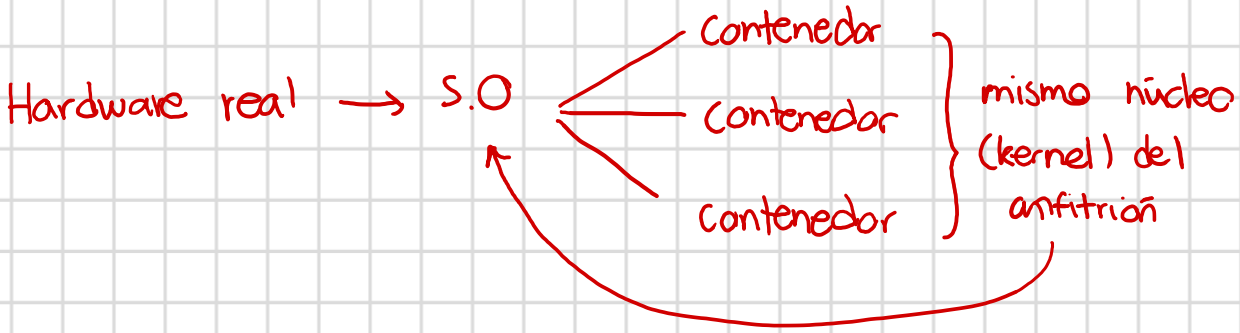
Paravirtualización: se modifica el S.O invitado para que "sepa" que es virtual y colabore con el hipervisor

Hardware real → Hipervisor → S.O invitado (modificado)

Colabora

Created with Notewise

Contenedores: no emula el hardware, sino el SO



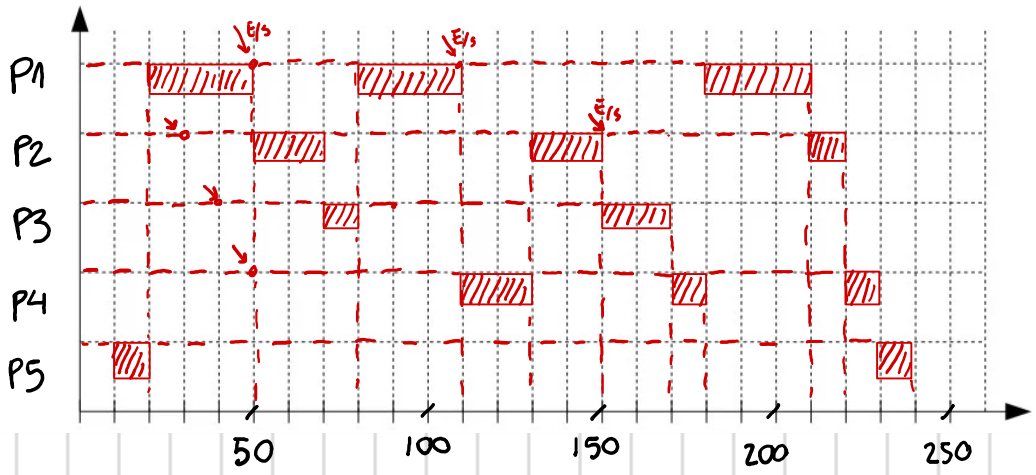
Pregunta 2 (3 puntos)

Un sistema utiliza prioridades fijas como algoritmo de planificación. En caso de procesos con igual prioridad se usa *Round Robin* (RR) con un cuanto de 20 u.t. Sabiendo que el sistema tiene que ejecutar los procesos que aparecen en la tabla, se pide:

1. (1 p) Dibujar el diagrama de Gantt de la ejecución de procesos.
2. (2 p) Explicar detalladamente el motivo de los cambios de proceso.

	T_{lleg}	Prio	T_{cpu} 1	$T_{E/S}$ 1	T_{cpu} 2	$T_{E/S}$ 2	T_{cpu} 3
P1	20	10	30	30	30	70	30
P2	30	15	40	20	10		
P3	40	15	30				
P4	50	15	40				
P5	10	20	20				

Espacio cuadriculado disponible por si fuera de utilidad para responder:



Prioridad	Proceso
10	P1 → $T_{E/S}$ 1 (30) → P1 → $T_{E/S}$ 2 (70) → P1 (30)
15	P2 → P3 → P4 → P2 (20) → P3 (20) → P4 (20) → $T_{E/S}$ P2 (20) → P2 (10) → P4 (10)
20	P5 → P5 (10)

Created with Notewise

Explicación cambios:

1er: Llega el P5 en el instante 10, pero en el 20 llega P1 (proceso de mayor prioridad)

2°: P1 acaba su ejecución de $T_{CPU\,1}$ y entra en $T_{E/S\,1}$, por lo que entra el siguiente proceso de mayor prioridad. Como en la siguiente prioridad hay varios procesos se desempata con la llegada de estos (aplicamos RR cuanto=2u) entrando así P2

3°: Acaba su cuanto y da paso a P3 el siguiente en la misma lista de prioridad

4°: Se está ejecutando P3, pero $T_{E/S\,1}$ de P1 a terminado por lo que antes de que acabe el cuanto de P3, es remplazado por P1 nuevamente

5°: P1 termina de ejecutar su $T_{CPU\,2}$ y vuelve a entra en $T_{E/S\,2}$, por lo que entra el siguiente proceso de mayor prioridad. En este caso es P4 (misma lista que P2, P3)

6°: P4 termina su cuanto y da paso nuevamente a P2

7°: P2 termina su cuanto y entra en $T_{E/S}$. Pasa P3

8°: P3 termina su cuanto y da paso a P4

9°: P1 termina su $T_{E/S\,2}$ y remplaza a P4 antes de que se termine su cuanto

Created with Notewise

10°: P1 termina de ejecutar su $T_{CPU\,2}$ y cambia al proceso de siguiente prioridad (P1 ya ha finalizado)

11°: Se le vuelve a dar paso a P2 que ya ha terminado de su $T_{E/S}$ y finaliza su tiempo de ejecución antes de que termine su cuanto (P2 ha finalizado). Se le da paso a P3

12°: De la misma manera que en el cambio anterior P3 termina su T_{CPU} antes de que acabe el cuanto (P3 ha finalizado). Se le da paso P4

13°: Sucede lo mismo que en P2 y P3 (P4 ha finalizado). Se le da paso a P5 (proceso de la prioridad más baja) que se termina de ejecutar y finaliza

Pregunta 3 (3 puntos)

Responder a las siguientes cuestiones:

1. (1 p) ¿Por qué motivos se cambia de proceso en ejecución? Detalla qué sucede a nivel de sistema operativo cada vez que se decide si se cambia de proceso.
2. (1 p) ¿Qué ventajas tiene usar múltiples hilos?

1) Se cambia de proceso en ejecución por diferentes motivos:

- Finaliza la ejecución: el proceso a llegado a su fin.
- Excepciones: el proceso tiene errores en el código o intenta ejecutar instrucciones prohibidas
- Interrupciones: hardware externo manda una señal para que el S.O recupere el control de la CPU (timer, que decide que la ejecución del programa a llegado a su fin)

Created with Notewise

Cada vez que se cambia de proceso sucede lo siguiente:

- Guardar el contexto: se guarda en la BCP la información para no perder el progreso
- Actuar: ejecuta el código necesario
- Decidir: se evalua si se devuelve la CPU al proceso o se cambia de proceso
- Planificador: decide que proceso es el siguiente de la lista de listos.
- Despachador: este es el componente que materializa el cambio de proceso

2) Usar multihilo mejora el rendimiento y la capacidad de respuesta de un programa, ya que varias hilos pueden ejecutar tareas de manera concurrente y compartir recursos dentro de un mismo proceso.
Si la CPU contiene multinúcleos se pueden realizar tareas al mismo tiempo, esto se le conoce como paralelismo

Created with Notewise

3. (1 p) Explica todas las posibles salidas por pantalla que produciría el siguiente programa. Utiliza un diagrama como los vistos en clase para respaldar las explicaciones.

```
#include <iostream>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdlib.h>
#include <pthread.h>

using namespace std;

int j = 10;

void *hello(void *arg)
{
   j++;
   cout << "hello " << getpid() << "-" << pthread_self()
        << ": j = " << j << endl;
   pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
   int r;
   r = fork();
   if (r < 0) {
      cout << "main1 " << getpid() << "-" << pthread_self()
           << ": j = " << j << endl;
      exit(1);
   }
   if (r == 0) {
      j++;
      cout << "main2 " << getpid() << "-" << pthread_self()
           << ": j = " << j << endl;
      pthread_t p;
      cout << "main3 " << getpid() << "-" << pthread_self()
           << ": j = " << j << endl;
      pthread_create(&p, NULL, hello, NULL);
      pthread_join(p, NULL);
      cout << "main4 " << getpid() << "-" << pthread_self()
           << ": j = " << j << endl;
      exit(0);
   }
   if (wait(&r) < 0) {
      cout << "main5 " << getpid() << "-" << pthread_self()
           << ": j = " << j << endl;
      exit(1);
   }
   cout << "main6 " << getpid() << "-" << pthread_self()
        << ": j = " << j << endl;
   exit(0);
}
```

Salidas:

main2 PID_hijo - hilo principal hijo : j = 11

main3 PID_hijo - hilo principal hijo : j = 11

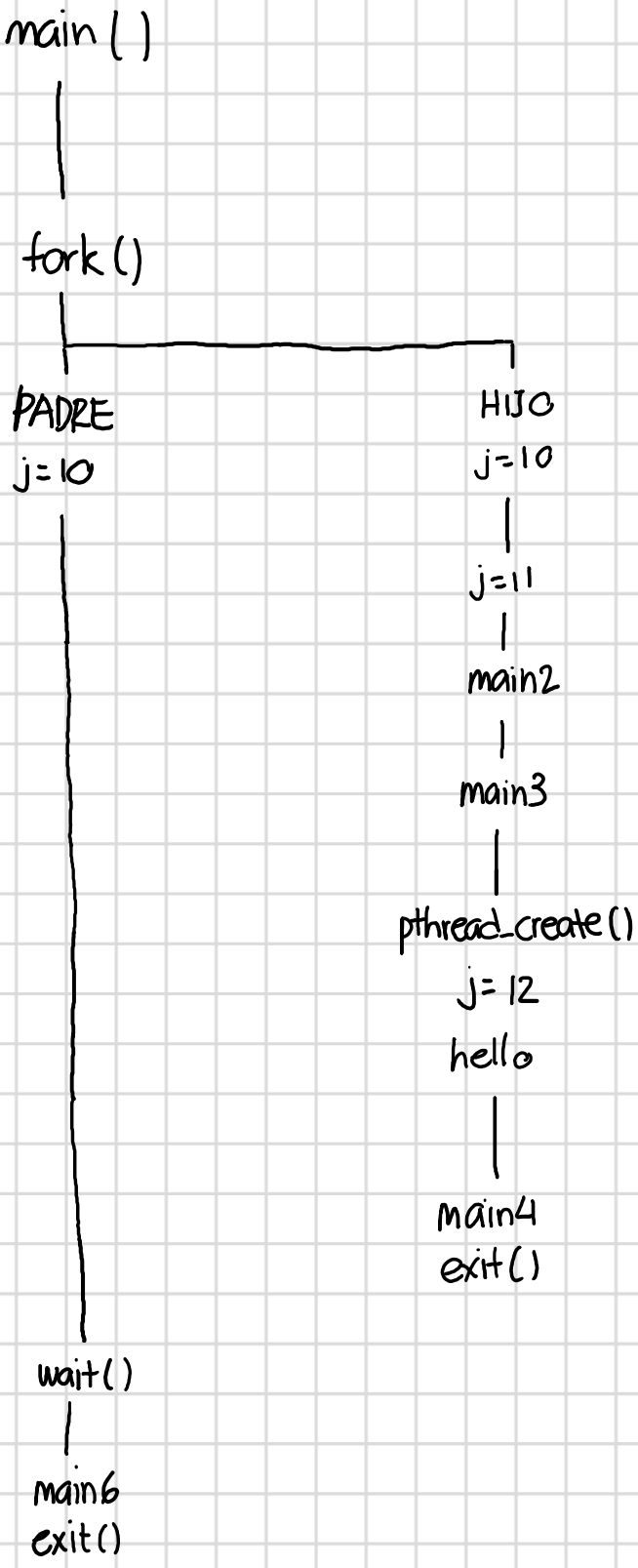
hello PID_hijo - hilo principal p : j = 12

main4 PID_hijo - hilo principal hijo : j = 12

main6 PID_padre - hilo principal padre : j = 10

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Created with Notewise



Created with Notewise